ACADEMIC PROJECT MASTER STUDY GUIDE

AI-Based Patient Appointment System

Complete Viva & Evaluation Manual for Milestone 1, Milestone 2, and Milestone 3

PROJECT DOMAIN
Healthcare AI & Full-Stack Web App

TECHNOLOGY ARCHITECTURE
React 18 + FastAPI + Python NLP + SQLite

TEAM MEMBERS / AUTHORS
Lokendra Prajapati (BTIT2603D03)
Samyak Lariya (BTIT2603D05)

ACADEMIC DETAILS
B.Tech Information Technology (B.Tech IT)
Academic Year: 2025–2026

1. Project Overview & Quick Elevator Pitch

Project One-Liner for Viva:
*"Our project is an **AI-Based Patient Appointment System** that bridges the gap between medical symptom self-assessment and doctor appointment scheduling. It features a 24/7 conversational AI triage chatbot that analyzes patient symptoms, detects medical emergencies, recommends the exact medical specialist (Cardiologist, Neurologist, etc.), prevents double-booking through atomic scheduling, and provides doctors with AI-generated pre-consultation clinical summaries."*

12+	< 1.5s	100%
MEDICAL SPECIALTIES	AI TRIAGE LATENCY	CONFLICT-FREE SLOTS

High-Level Problem It Solves

- **Patient Self-Diagnosis Problem:** Patients often don't know which doctor to consult (e.g. visiting an Orthopedist for joint pain caused by Rheumatic fever or visiting a General Physician for acute neurological symptoms), leading to wasted appointments and delayed treatment.
- **Disconnection in Current Apps:** Apps like Practo/Zocdoc book appointments but don't do AI symptom diagnosis. Symptom checkers like Ada Health diagnose symptoms but don't book nearby doctors. Our system unifies both!
- **Doctor Context Deficiency:** Doctors currently spend 10–15 minutes asking basic preliminary questions. Our system generates a ready-to-read pre-consultation summary before the patient walks in.
- **Emergency Overrides:** Patients with acute life-threatening symptoms (heart attack, stroke) are immediately redirected to Emergency Services (ER) rather than booking normal clinic slots.

🎯 **VIVA Q: How will you introduce the project to the examiner in 30 seconds?**
*"Good morning Sir/Ma'am. Our project is an **AI-Based Patient Appointment System**. In existing platforms, patients either self-diagnose and choose the wrong doctor on booking apps like Practo, or they use symptom checkers like Ada Health without direct appointment booking. Our platform combines conversational AI symptom triage, emergency red-flag detection, automatic doctor-specialty recommendation, conflict-free real-time slot booking, and pre-consultation AI clinical summaries for doctors on a unified, secure dashboard."*

2. Summary of Milestones (1, 2, and 3)

Here is what was accomplished across each milestone so far:

Milestone	Key Objective & Scope	What Was Built / Done	Key Deliverables / Tools
Milestone 1 STUDY & REQUIREMENT	Literature review, comparative study of existing healthcare systems, gap analysis, FRs & NFRs.	• Analyzed Practo, Zocdoc, Ada Health, Babylon Health, Buoy Health. • Identified 3 primary industry gaps. • Defined 5 Functional & 4 Non-Functional Requirements.	• Comparative Matrix Table • Problem Statement • Requirement Specification Document
Milestone 2 DATA & AI ENGINE	Dataset collection, clinical preprocessing, exploratory data analysis (EDA), and 2-tier AI triage engine.	• Curated symptom-disease-specialist knowledge base (CUMC, Mayo Clinic, ICD-10). • 6-stage data preprocessing pipeline (cleaning, tokenization, synonym mapping). • 2-Tier AI Engine (Emergency classifier + Jaccard similarity scoring).	• `symptom_specialist_dataset.csv` • `ai_engine.py` backend module • EDA urgency & specialty distribution charts
Milestone 3 SYSTEM ARCHITECTURE	Comprehensive system architecture, 8 functional subsystem modules, ER database design, security & API flows.	• Designed 4-tier decoupled client-server architecture. • Decomposed 8 core modules (Auth, Chatbot, AI Triage, Doctor Directory, Booking, Records, Dashboards). • Built normalized 3NF Relational DB schema (SQLAlchemy). • Google OAuth 2.0 GIS + JWT + OTP integration.	• High-Level Architecture Diagram • ER Database Diagrams • REST API Endpoints Specs • Safety & Medical Disclaimers

3. Deep-Dive: Milestone 1 (Existing Systems & Gap Analysis)

3.1 Comparative Analysis Table

System	Category	AI Capabilities	Key Strength	Major Limitation / Gap
Practo	Doctor Booking	Minimal (Keyword search)	Large doctor network	Patients must self-diagnose doctor specialty.
Zocdoc	Doctor Discovery	Basic NLP search	Real-time calendar UI	No symptom triage; user needs prior medical awareness.
Ada Health	AI Symptom Triage	High (Bayesian Network)	Deep symptom assessment	**No appointment booking** or doctor directory.
Babylon Health	Telehealth & Triage	High (NLP + Rules)	Combines chat & video	High operational complexity; no local clinic slot locks.
Buoy Health	Navigation Care	Moderate-High	ER vs Routine triage	Fragmented third-party provider handoff.

3.2 Three Major Gaps Addressed

1. **Assessment-to-Booking Disconnect:** Existing platforms excel at *either* symptom assessment (Ada) *or* booking (Practo), but do not unite them.
2. **Self-Diagnosis Pitfall:** Patients routinely book the wrong specialist, leading to delayed medical care and consultation cancellation.
3. **Doctor Context Void:** Doctors start consultations with zero structured pre-consultation insights.

3.3 Requirements Formulation

Functional Requirements (FR)

- **FR-1:** Patient & Doctor Auth (Google OAuth, JWT, OTP)
- **FR-2:** AI Symptom Chatbot Drawer
- **FR-3:** Smart Specialist Recommendation Engine
- **FR-4:** Real-time Conflict-Free Slot Booking
- **FR-5:** Dual Dashboards (Patient & Doctor)

Non-Functional Requirements (NFR)

- **NFR-1 Security:** AES-256 data at rest, TLS 1.3 in transit, Bcrypt.
- **NFR-2 Usability:** Responsive SPA UI, sub-second responses.
- **NFR-3 Performance:** AI response < 1.5s, API booking < 500ms.
- **NFR-4 Safety:** Prominent clinical disclaimers & ER override.

🎯 VIVA Q: Why not just use Practo or Google to search symptoms?

"Searching symptoms on Google causes 'cyberchondria' (unnecessary panic) and often gives misleading suggestions. Practo requires the patient to know beforehand if they need an Endocrinologist, Neurologist, or Rheumatologist. Our system uses clinical token matching and red-flag rules to guide patients to the correct doctor safely in one continuous flow."

4. Deep-Dive: Milestone 2 (Data Collection, EDA & AI Engine)

4.1 Dataset Sourcing & Taxonomy

The medical knowledge base was synthesized from authoritative clinical repositories: **Columbia University Medical Center (CUMC) Disease-Symptom Knowledge Graph**, **Mayo Clinic & NHS Clinical Triage Rules**, and **ICD-10 Categorization Guidelines**.

4.2 6-Stage Data Preprocessing Pipeline

```
[Raw Clinical Symptom Data & User Input Text]
       |
       ▼  Step 1: UTF-8 Encoding & Ingestion Sanitization
       ▼  Step 2: Null / Incomplete Record Pruning
       ▼  Step 3: Deduplication & Semantic Redundancy Removal
       ▼  Step 4: Clinical NLP Tokenization & Lowercasing (Regex \w+)
       ▼  Step 5: Medical Synonym Harmonization (e.g., "dyspnea" → "shortness of breath")
       ▼  Step 6: Categorical Severity Stratification (Emergency / High / Med / Low)
       |
[Validated Clinical Knowledge Base → Fed into backend/app/services/ai_engine.py]
```

4.3 The 2-Tier AI Recommendation Algorithm

Tier 1: Emergency Classifier (Red-Flag Filter)
High-priority token set: `["chest pain", "shortness of breath", "slurred speech", "numbness in face", "arm pain", "stroke", "heart attack", "unconscious", "severe bleeding", "difficulty breathing"]`.
If any emergency token is present → Triggers instant **EMERGENCY WARNING**, instructs the patient to call emergency services (112/108/911), and bypasses elective clinic booking for patient safety.

Tier 2: Tokenized Jaccard Similarity Matcher & Confidence Calculation
For non-emergency inputs, the AI engine tokenizes the user input tokens U and dataset symptom tokens S: $$\text{Similarity Score} = \frac{|U \cap S|}{\max(|S|, 1)}$$ $$\text{Confidence} = \min\left(0.60 + (\text{Similarity Score} \times 0.35),\, 0.95\right)$$ **Safe Default Fallback:** If similarity score across all 12 specialties is ≤ 0.15, it routes safely to a **General Physician** with baseline confidence of **70%**.

4.4 Medical Specialties & Urgency Stratification

Specialty	Example Conditions Mapped	Urgency Tier	Emergency Flag
Cardiologist	Heart Attack, Angina / Arrhythmia	HIGH / CRITICAL	`True`
Neurologist	Stroke, Migraine, Sciatica	HIGH (STROKE) / MED	`True (Stroke)`
Pulmonologist	Asthma, Pneumonia, Bronchitis	MEDIUM	`False`
Dermatologist	Allergic Dermatitis, Eczema, Acne	LOW / ROUTINE	`False`
Orthopedist	Osteoarthritis, Knee/Joint pain	MEDIUM	`False`
Endocrinologist	Diabetes Mellitus, Hypothyroidism	MED / LOW	`False`
General Surgeon	Acute Appendicitis	HIGH	`True`
ENT Specialist	Otitis Media, Tonsillitis	LOW	`False`
General Physician	General Viral Fever, Body Aches, Fatigue	LOW / MED	`False`

🎯 **VIVA Q: What algorithm did you use in Milestone 2 for symptom assessment?**

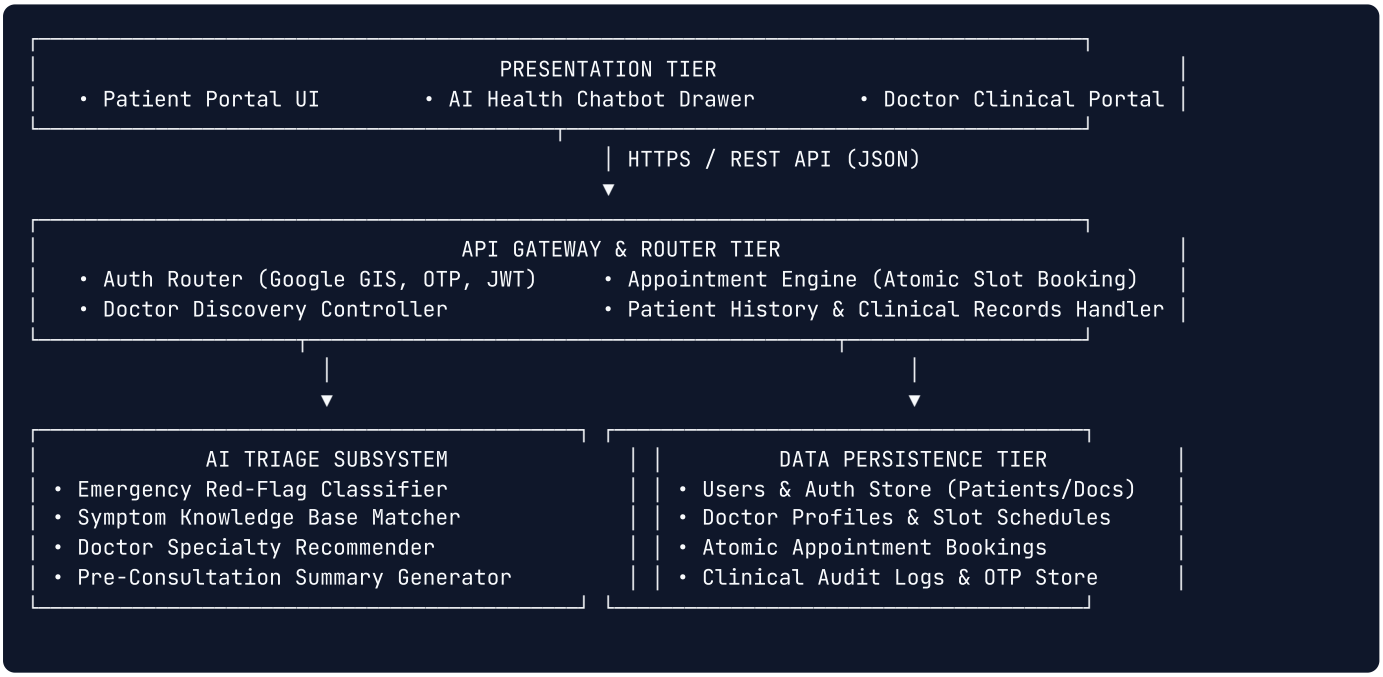
*"We built a **Two-Tier Hybrid AI Triage Engine** in Python. Tier 1 executes deterministic emergency keyword detection (for red flags like chest pain or slurred speech) to immediately warn patients and redirect to emergency care. Tier 2 computes tokenized Jaccard similarity and overlap scoring against a normalized clinical knowledge base to recommend the exact specialist with confidence between 60% and 95%. It includes a safe fallback to General Physician if similarity is below threshold."*

5. Deep-Dive: Milestone 3 (System Architecture & Module Design)

5.1 Technology Stack Selection & Rationale

Layer	Selected Tech	Why We Chose It (Viva Explanation)
Frontend	**React 18 + Vite / Modern SPA**	Sub-second load time, component modularity (ChatbotDrawer, AppointmentModal), responsive glassmorphism UI.
Backend API	**FastAPI (Python 3.10+)**	Async (`asyncio`) performance, automated Swagger/OpenAPI docs, native Pydantic v2 data validation.
AI Triage	**Python NLP Engine + Rules**	Fast execution (< 50ms processing time), zero external API cost/failure, deterministic emergency safety.
Database & ORM	**SQLAlchemy ORM + SQLite / Postgres**	Relational integrity, foreign key cascading, ACID transactions preventing double-booking race conditions.
Authentication	**Google OAuth 2.0 (GIS) + JWT + Bcrypt**	One-click Google Sign-In with Google Identity Services, cryptographic JWT tokens, Bcrypt password hashing.

5.2 4-Tier Decoupled System Architecture



5.3 The 8 Subsystem Modules

1. Auth & Identity Service
Google OAuth 2.0 GIS token verification, OTP generator (SMS/Email), Bcrypt password hashing, JWT access/refresh tokens.

2. AI Chatbot Drawer
Interactive drawer sliding in from the right, multi-turn conversational symptom intake, instant safety badge rendering.

3. Clinical Assessment Engine
Parses unstructured text, scores against 20+ disease clusters, detects emergency conditions, calculates confidence level.

4. Doctor Recommendation Service
Filters verified doctors matching the diagnosed specialty, sorted by rating, fees, and location proximity.

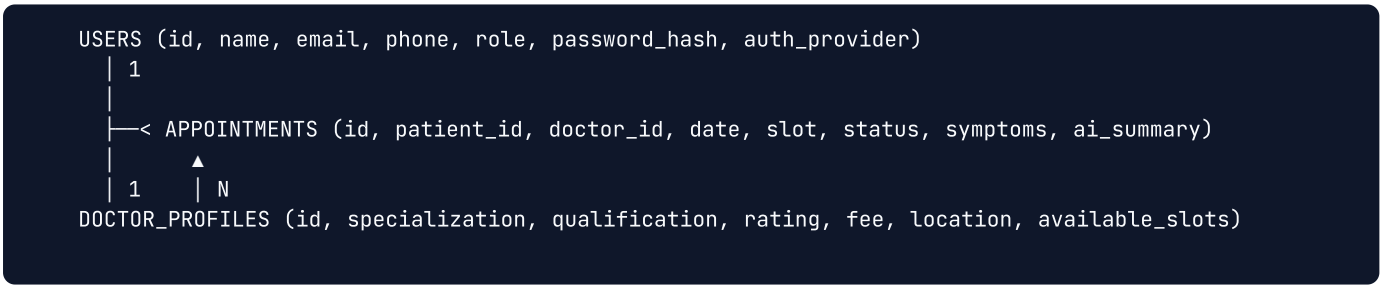
5. Doctor Schedule Management
Granular 30-minute consultation slots, dynamic availability checks, doctor leave handling.

6. Appointment Booking Engine
Atomic transaction booking, duplicate slot prevention, unique appointment tracking IDs.

7. Patient Health Records
Patient consultation history, symptoms logged, AI assessment history, active/past status.

8. Doctor Clinical Dashboard
Live queue of scheduled patients, AI-generated clinical pre-briefings, consultation status toggles.

5.4 Database Entity-Relationship (ER) Design



🎯 VIVA Q: How does your system prevent two patients from booking the same doctor slot?

*"We enforce **Atomic Slot Locking** in the backend booking service and database constraints. When an appointment request is received, the backend checks for existing active appointments matching `(doctor_id, date, slot)`. If found, it rejects the duplicate transaction with an HTTP 409 Conflict error. In the UI, booked slots are dynamically disabled in real time."*

6. Master Viva Questions & Answers (Top 15 Expected Questions)

🎯 VIVA Q: Q1: What is the main objective of your project?

Answer: The main objective is to provide an end-to-end intelligent healthcare platform that uses AI to analyze patient symptoms, detect emergencies, recommend the right medical specialist, enable seamless real-time appointment booking without slot conflicts, and provide doctors with structured AI clinical summaries before consultation.

🎯 VIVA Q: Q2: What is the difference between Milestone 1, 2, and 3?

Answer:

- **Milestone 1:** Problem definition, market study of existing systems (Practo, Ada), identifying research gaps, and writing Functional/Non-Functional Requirements.
- **Milestone 2:** Dataset collection from clinical taxonomies, 6-stage data preprocessing, exploratory data analysis (EDA), and creating the 2-tier AI triage engine.
- **Milestone 3:** Full system architecture, technology stack selection, 8-module decomposition, relational ER database modeling, and Google OAuth 2.0 / JWT security design.

🎯 VIVA Q: Q3: Which technologies are you using across the stack and why?

Answer:

- **Frontend:** React 18 + Vite for high performance, modular components, and responsive modern UI.
- **Backend:** FastAPI (Python) because it is asynchronous (`asyncio`), ultra-fast, has built-in Pydantic validation, and auto-generates Swagger API documentation.
- **Database:** SQLite for development and PostgreSQL for production with SQLAlchemy ORM for relational integrity.
- **AI Engine:** Custom Python NLP triage module using tokenized Jaccard similarity and deterministic red-flag emergency rule matching.
- **Authentication:** Google OAuth 2.0 (Google Identity Services) + JWT tokens + Bcrypt for password security.

🎯 VIVA Q: Q4: How does your AI triage handle medical emergencies?

Answer: We have a dedicated **Tier 1 Emergency Classifier** that checks for critical red-flag keywords (such as severe chest pain, shortness of breath, slurred speech, sudden facial numbness). When detected, it immediately displays an **Emergency Red Alert** instructing the patient to call emergency services (112/108/911) or visit the nearest ER immediately, completely overriding elective clinic booking.

🎯 VIVA Q: Q5: How does the AI calculate confidence for doctor recommendations?

Answer: We calculate the token overlap (Jaccard similarity) between user symptom tokens and our validated clinical knowledge base. The confidence score is mapped dynamically between **60% and 95%**: $\text{Confidence} = \min(0.60 + (\text{Similarity} \times 0.35), 0.95)$. If the similarity is ≤ 0.15, it routes safely to a **General Physician** with 70% confidence.

🎯 VIVA Q: Q6: What happens if the patient types symptoms that don't match any specialist?

Answer: The system has a safe fallback mechanism. If no specific specialist matches above our threshold (≤ 15% similarity), the AI routes the patient to a **General Physician** (Internal Medicine), which is the standard medical protocol for non-specific symptoms.

🎯 VIVA Q: Q7: How is patient safety and legal compliance addressed?

Answer:

1. Every AI assessment contains a statutory disclaimer: *"This is a preliminary AI triage assessment and does not constitute a definitive medical diagnosis or prescription."*
2. Doctor-in-the-Loop: All AI assessments are provided to the doctor as a draft summary for clinical verification.
3. Data privacy: Encrypted transit (TLS) and rest (AES-256) with role-based access control (RBAC).

🎯 VIVA Q: Q8: How does the Google Sign-In authentication work?

Answer: We use **Google Identity Services (GIS) OAuth 2.0**. The frontend receives a signed Google ID credential (JWT) from Google and sends it to our backend endpoint `/api/auth/google`. The backend verifies the cryptographic signature with Google's public certificates, extracts the user's email, name, and profile, and creates/retrieves the user in our database while issuing our own secure JWT session token.

🎯 VIVA Q: Q9: How do doctors benefit from your system?

Answer: Doctors get a dedicated **Doctor Dashboard** showing their daily appointment queue, patient details, and most importantly, an **AI-generated pre-consultation summary** of reported symptoms and urgency level. This saves 5–10 minutes of manual inquiry per patient and allows doctors to focus on clinical examination.

🎯 VIVA Q: Q10: What are the future milestones (Milestones 4 & 5)?

Answer:

- **Milestone 4:** Complete frontend and backend implementation, interactive chatbot UI, slot booking modal, and doctor dashboard.
- **Milestone 5:** Comprehensive system testing, security audit, latency benchmarking, edge case validation, and final project deployment.